

Precious Jackson

CS 499

9/23/2026

Milestone Three: Enhancement Two – Algorithms and Data Structures

Artifact Description

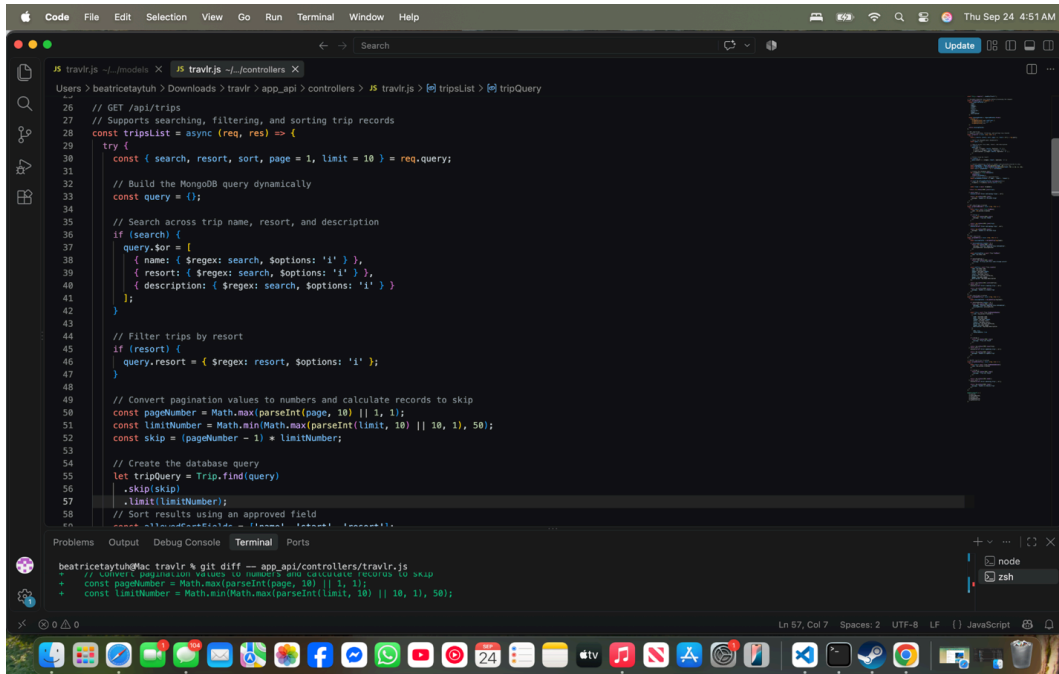
The artifact I selected for this enhancement is Travlr Getaways, a full-stack web application that I originally developed in June 2026 during CS 465: Full Stack Development. The application was designed as a travel website that allows trip information to be stored in a MongoDB database and retrieved through an API. The application was designed as a travel website that allows trip information to be stored in a MongoDB database and retrieved through an API. The project uses technologies including Node.js, Express, MongoDB, and Mongoose to manage and process trip information.

I selected Travlr Getaways for my ePortfolio because it provides an opportunity to demonstrate how algorithms and data-processing techniques can improve the way an application retrieves and organizes information. The original application could retrieve trip records, but the enhancement expanded the API's ability to search, filter, sort, and process those records. I also added pagination to control how many records are returned at one time and to support more efficient processing as the amount of trip data grows.

Justification for Inclusion and Skills Demonstrated

I included Travlr Getaways in my ePortfolio because the enhancement demonstrates my ability to apply algorithmic thinking to a practical full-stack application. Instead of simply retrieving all trip records in the same manner, the enhanced tripsList controller dynamically builds a query based on information supplied by the user.

The search functionality evaluates multiple fields, including the trip name, resort, and description. I used an array of search conditions with MongoDB's \$or operator so that one search value can be evaluated against multiple parts of a trip record. The resort filter further narrows the result set, while the sorting logic restricts sorting to approved fields such as name, start date, and resort. Limiting the available sorting fields also gives the application more control over how incoming query parameters are processed.



```
26 // GET /api/trips
27 // Supports searching, filtering, and sorting trip records
28 const tripsList = async (req, res) => {
29   try {
30     const { search, resort, sort, page = 1, limit = 10 } = req.query;
31     // Build the MongoDB query dynamically
32     const query = {};
33     // Search across trip name, resort, and description
34     if (search) {
35       query.$or = [
36         { name: { $regex: search, $options: 'i' } },
37         { resort: { $regex: search, $options: 'i' } },
38         { description: { $regex: search, $options: 'i' } }
39       ];
40     }
41     // Filter trips by resort
42     if (resort) {
43       query.resort = { $regex: resort, $options: 'i' };
44     }
45     // Convert pagination values to numbers and calculate records to skip
46     const pageNumber = Math.max(parseInt(page), 1) || 1;
47     const limitNumber = Math.min(Math.max(parseInt(limit), 10) || 10, 1, 50);
48     const skip = (pageNumber - 1) * limitNumber;
49     // Create the database query
50     let tripQuery = Trip.find(query)
51       .skip(skip)
52       .limit(limitNumber);
53     // Sort results using an approved field
54     const allowedSortFields = ['name', 'start', 'resort'];
55     if (sort && allowedSortFields.includes(sort)) {
56       tripQuery = tripQuery.sort(sort);
57     }
58     const trips = await tripQuery.exec();
59     res.json(trips);
60   } catch (error) {
61     res.status(500).json({ message: 'Server error' });
62   }
63 }
```

Figure 1. Enhanced Trip Search, Filtering, Sorting, and Pagination Logic.

The enhanced tripsList controller dynamically processes trip records using search, filtering, sorting, and pagination. Search criteria are evaluated across multiple trip fields, while

pagination calculates an offset based on the requested page and limits the number of records returned. The page-size safeguard also restricts requests to a maximum of 50 records, supporting more controlled and scalable data processing.

One of the most significant improvements was the addition of pagination. The controller accepts page and limit parameters and converts them into numeric values before calculating the number of records that should be skipped. The calculation $(\text{pageNumber} - 1) * \text{limitNumber}$ determines the starting position for each requested page. The result is then applied to the database query through `skip()` and `limit()`.

I also established a maximum page size of 50 records. This represents a design trade-off between providing users with enough information and preventing a single request from retrieving an unnecessarily large result set. Default values also allow the API to continue functioning when page and limit parameters are not supplied.

Testing showed that these operations can work together rather than functioning only as isolated features. For example, I submitted a request that searched for records containing "reef," sorted the matching records alphabetically, and limited the first page to two results. The API correctly returned Claire's Reef followed by Dawson's Reef.

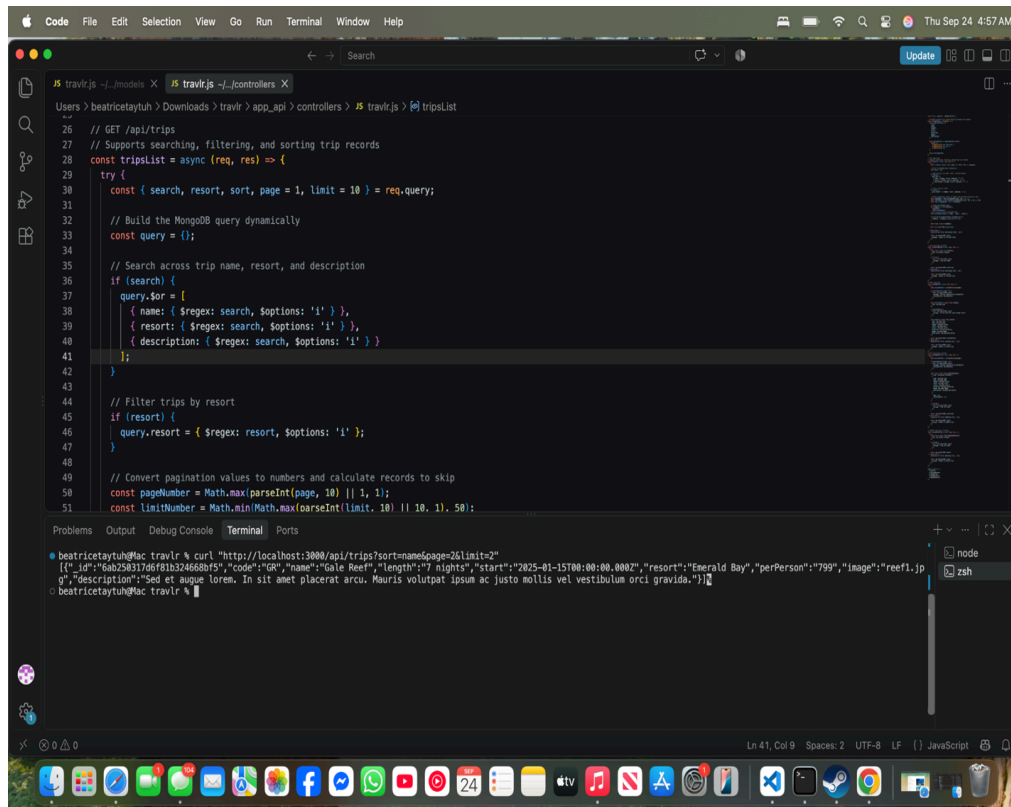
```
26 // GET /api/trips
27 // Supports searching, filtering, and sorting trip records
28 const tripsList = async (req, res) => {
29   try {
30     const { search, resort, sort, page = 1, limit = 10 } = req.query;
31
32     // Build the MongoDB query dynamically
33     const query = {};
34
35     // Search across trip name, resort, and description
36     if (search) {
37       query.$or = [
38         { name: { $regex: search, $options: 'i' } },
39         { resort: { $regex: search, $options: 'i' } },
40         { description: { $regex: search, $options: 'i' } }
41       ];
42     }
43
44     // Filter trips by resort
45     if (resort) {
46       query.resort = { $regex: resort, $options: 'i' };
47     }
48
49     // Convert pagination values to numbers and calculate records to skip
50     const pageNumber = Math.max(parseInt(page, 10) || 1, 1);
51     const limitNumber = Math.min(Math.max(parseInt(limit, 10) || 10, 1), 50);
```

```
beatricetayuh@Mac travlr % curl "http://localhost:3000/api/trips?search=reef&sort=name&page=1&limit=2"
[{"_id":"6ab258317d6f81b3246680f4","code":"CR","name":"Claire's Reef","length":"10 nights","start":"2025-03-10T00:00:00.000Z","resort":"Coral Point","person":"999","image":"reef1.jpg","description":"Donec sed felis risus. Nulla facilisi. Donec a orci tellus."},{ "_id":"6ab258317d6f81b3246680f2","code":"DR","name":"Dawson's Reef","length":"6 nights","start":"2025-02-20T00:00:00.000Z","resort":"Blue Lagoon","person":"999","image":"reef2.jpg","description":"Integer magna leo, posuere et dignissim vitae, porttitor at odio."}]
beatricetayuh@Mac travlr %
```

Figure 2. Combined Search, Sorting, and Pagination Test.

The enhanced Trips API was tested with a combined request that searches for records containing "reef," sorts the matching trips alphabetically by name, and limits the first page to two results. The API returned Claire's Reef followed by Dawson's Reef, demonstrating that the search, sorting, and pagination operations function together as intended.

I then requested the second page with the same two-record limit. Because the first two records were skipped, the API returned the remaining Gale Reef record. This test demonstrated that the pagination calculation correctly determines which portion of the result set should be retrieved.



```
travlr.js - ./models X  travlr.js - ./controllers X
Users > beatrixtaylor@Downloads: travlr > app_api > controllers > travlr.js > tripsList
26 // GET /api/trips
27 // Supports searching, filtering, and sorting trip records
28 const tripsList = async (req, res) => {
29   try {
30     const { search, resort, sort, page = 1, limit = 10 } = req.query;
31
32     // Build the MongoDB query dynamically
33     const query = {};
34
35     // Search across trip name, resort, and description
36     if (search) {
37       query.$or = [
38         { name: { $regex: search, $options: 'i' } },
39         { resort: { $regex: search, $options: 'i' } },
40         { description: { $regex: search, $options: 'i' } }
41       ];
42     }
43
44     // Filter trips by resort
45     if (resort) {
46       query.resort = { $regex: resort, $options: 'i' };
47     }
48
49     // Convert pagination values to numbers and calculate records to skip
50     const pageNumber = Math.max(parseInt(page, 10) || 1, 1);
51     const limitNumber = Math.min(Math.max(parseInt(limit, 10) || 10, 1), 50);
52
53     const trips = await Trip.find(query).sort(sort).skip((pageNumber - 1) * limitNumber).limit(limitNumber);
54     const totalTrips = await Trip.countDocuments(query);
55     res.json(trips);
56   } catch (error) {
57     res.status(500).json({ message: 'Internal server error' });
58   }
59 }
56
Problems Output Debug Console Terminal Ports
beatrixtaylor@Mac travlr % curl "http://localhost:3000/api/trips?sort=name&page=2&limit=2"
[{"_id":"6ab258317d6f81b324668b5","code":"GR","name":"Gale Reef","length":"7 nights","start":"2025-01-15T00:00:00.000Z","resort":"Emerald Bay","perPerson":"799","image":"reef1.jpg","description":"Sed et augue lorem. In sit amet placerat arcu. Mauris volutpat ipsum ac justo mollis vel vestibulum orci gravida."}]
beatrixtaylor@Mac travlr %
```

Figure 3. Pagination Offset and Second-Page Retrieval.

The Trips API was tested by requesting the second page with a limit of two records while sorting by trip name. The pagination algorithm calculated the appropriate offset to skip the first two records, causing the API to return only the remaining Gale Reef record. This confirms that the page and limit parameters correctly control which portion of the trip collection is retrieved.

Filtering was also tested independently. A request specifying Blue Lagoon as the resort returned only Dawson's Reef, confirming that the API correctly narrows the result set according to the supplied filtering criteria.

```
travlr.js - ./models X travlr.js - ./controllers X
Users > beatricetayuh > Downloads > travlr > app_api > controllers > JS travlr.js > @tripsList
26 // GET /api/trips
27 // Supports searching, filtering, and sorting trip records
28 const tripsList = async (req, res) => {
29   try {
30     const { search, resort, sort, page = 1, limit = 10 } = req.query;
31
32     // Build the MongoDB query dynamically
33     const query = {};
34
35     // Search across trip name, resort, and description
36     if (search) {
37       query.$or = [
38         { name: { $regex: search, $options: 'i' } },
39         { resort: { $regex: search, $options: 'i' } },
40         { description: { $regex: search, $options: 'i' } }
41       ];
42     }
43
44     // Filter trips by resort
45     if (resort) {
46       query.resort = { $regex: resort, $options: 'i' };
47     }
48
49     // Convert pagination values to numbers and calculate records to skip
50     const pageNumber = Math.max(parseInt(page, 10) || 1, 1);
51     const limitNumber = Math.min(Math.max(parseInt(limit, 10) || 10, 1), 50);
52   } catch (error) {
53     res.status(500).json({ message: error.message });
54   }
55   // ... (rest of the function) ...
56 }
```

```
beatricetayuh@Mac travlr % curl "http://localhost:3000/api/trips?resort=BlueLagoon"
[{"_id":"6ab258317d9fd8324668b73","code":"DR","name":"Dawson's Reef","length":"6 nights","start":"2025-02-20T00:00:00.000Z","resort":"Blue Lagoon","perPerson":"699","image":"reef2.jpg","description":"Integer magna leo, posuere et dignissim vitae, porttitor at odio."}]
beatricetayuh@Mac travlr %
```

Figure 4. Resort Filtering Test.

The enhanced Trips API was tested by filtering records for the Blue Lagoon resort. The request returned only Dawson's Reef, demonstrating that the dynamic filtering logic correctly narrows the trip collection according to the specified resort.

Together, these improvements demonstrate my ability to use data structures, conditional processing, query construction, sorting, filtering, and pagination to solve a practical data-retrieval problem.

Course Outcomes

This enhancement allowed me to make substantial progress toward the course outcome of designing and evaluating computing solutions that solve a given problem using algorithmic principles and computer science practices while considering the trade-offs involved in design choices.

My original plan was to improve the searching, filtering, sorting, and processing of trip data in Travlr Getaways. I met that goal by implementing and testing dynamic search, resort filtering, controlled sorting, and pagination. I expanded the original plan through pagination because it provided an additional way to demonstrate how an application can process a collection of records without returning the entire result set for every request.

The enhancement also supports the program outcome involving the use of well-founded techniques, skills, and tools to implement computing solutions that deliver value and accomplish specific goals. The API provides more flexible ways to retrieve relevant information while maintaining control over the number of records processed in an individual request.

I also considered security and defensive programming when designing the enhancement. Sorting is restricted to an approved list of fields, and pagination values are normalized before being used. The page number cannot fall below one, and the number of records requested is restricted to a maximum of 50. These decisions provide more predictable handling of client-supplied query parameters.

I further strengthened the query-processing logic by sanitizing values used in regular-expression searches. Because the search and resort parameters originate from client requests, characters with special meaning in a regular expression could otherwise change the behavior of the intended search. I implemented an `escapeRegex` helper that escapes regular-expression metacharacters before the values are passed to MongoDB. I then tested the enhancement using normal and adversarial-style inputs. A search for reef continued to return the expected trip records, while

requests using `.*` as either the search or resort value returned no records instead of treating the characters as a wildcard expression. This enhancement demonstrates defensive programming by allowing flexible search functionality while maintaining greater control over how client-supplied input is interpreted.

I do not currently need to make a major change to my planned outcome coverage. The completed enhancement demonstrates the algorithms and data structures outcome as intended while also providing supporting evidence for software design, security-minded development, testing, and technical communication.

Reflection on the Enhancement Process

Enhancing Travlr Getaways helped me better understand that algorithms and data structures are not limited to complex mathematical algorithms or standalone programming exercises. They are also involved in everyday application decisions about how data is searched, organized, filtered, and divided into manageable sets.

One of the most useful parts of this enhancement was implementing pagination. Calculating the number of records to skip based on the requested page helped me connect a relatively simple mathematical calculation to a practical application feature. For example, when page two is requested with a limit of two records, the calculation determines that the first two records should be skipped before the next portion of the result set is returned.

I also learned the importance of testing features both individually and together. I first tested searching, filtering, and sorting separately to confirm that each operation behaved correctly. I then tested pagination using multiple pages and finally combined searching, sorting, and pagination within the same request. This approach made it easier to identify where a problem would occur if one part of the processing logic did not behave as expected.

A challenge during the enhancement was making sure that the new pagination functionality did not interfere with the search, filtering, and sorting behavior that was already working. I addressed this by making small changes to the existing controller and testing the application after each change rather than rewriting the entire function. This allowed me to preserve existing functionality while adding new processing capabilities.

Another important lesson was understanding the trade-off between flexibility and control. Allowing a client to choose a page size is useful, but allowing an unlimited value could result in unnecessarily large requests. Setting a maximum limit of 50 records provided a balance between flexibility and controlled resource use. Similarly, restricting sorting to approved fields makes the behavior of the API more predictable.

Another lesson from the enhancement process was the importance of considering how user-supplied input is interpreted by the application. Although the search and resort filtering features were functioning correctly with normal input, additional testing showed that regular-expression characters could affect how a search was processed. I addressed this by adding an `escapeRegex` helper that escapes special regular-expression characters before the

search and resort values are used in MongoDB queries. I then tested the updated logic with both normal search values and `.*`. The normal searches continued to return the expected records, while `.*` no longer behaved as a wildcard expression. This process helped me understand that successful functionality testing is only one part of evaluating an algorithm. I also need to consider unexpected input, security, and how the application will behave outside of normal use cases.

Overall, this enhancement strengthened my understanding of how algorithmic principles can be incorporated into full-stack development. The completed artifact demonstrates not only that I can implement search, filtering, sorting, and pagination, but also that I can evaluate how these operations interact, test edge cases, and make design decisions based on maintainability, scalability, and application requirements.